We think learning is most motivating and effective when you take the place of that original idea-maker, who was thinking of how to solve a particular problem. Only once the solution crystalizes does it warrant formalization, which is indeed necessary to establish its correctness and to faithfully communicate it to others in the field.

There is a powerful urge to engage in this reverse chronological mode of teaching, but we will do our best to resist it and develop the topic as we go. In that spirit, we will introduce new terms, definitions, and mathematical notations as we need them. For example, we will use “callouts” like this:

Definition

A *neural network* is a kind of machine learning model composed of multiple “layers” that perform a matrix-vector multiplication followed by the application of a nonlinear “activation” function. The matrices of the neural network are the model’s learnable parameters and are often called the “weights” of the neural network.

You will only see these callouts once per term, but we will often repeat the definition in different ways in the text to make sure you really understand and remember it. This is a course on reinforcement learning, not a textbook or reference, so we won’t shy away from repeating ourselves when we think something is important to remember.

Whenever we need to introduce some math, we will typically use a box showing the math and a pseudo-Python version of the same underlying concept. Sometimes it’s easier to think in terms of code or of math, and we think it’s good to get familiar with both. As a super simple example, if we were introducing the equation of a line, we would do it like this:

Table 2.1. Example of the side-by-side mathematics and pseudocode we use in this book